



University
Mohammed VI
Polytechnic



Deliverable #: Lab-5/Normalization and SQL Implementation

Data Management Course
UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering
Session: Fall 2025

Team Information

Team Name	StarsDB
Member 1	NouhaTalssi
Member 2	MariamSaid
Member 3	TilamsaneWissal
Member 4	AlaeSabir
Member 5	AminaSidki
Repository Link	https://github.com/StarsDB

1 -Introduction

This deliverable focuses on the normalization of the MNHS database schema and its implementation using SQL ,plus the implementation of some queries. The goal is to ensure data integrity, reduce redundancy, and optimize database performance.

2 -Requirements

In this deliverable, we are going to:

- Validate all the relations of the MNHS database schema against BCNF.
- Verify that all functional dependencies are preserved during normalization.
- Implement the normalized schema using SQL.
- Alter the table Patient and Staff to add new attributes (email and home address).
- Insert 5 rows of data into each table.
- Update the attributes phone number for the table patient and the attribute region for the attribute hospital.
- Delete a scheduled appointment from the table appointment.
- Write and execute SQL queries to retrieve specific information from the database.

3 -Methodology

- Query1: In order to have the second part of the full name (last name) we had to use a function called SUBSTRING_INDEX.
- Query2: We understand from The query “List distinct insurance types” we need to retrieve the different categories of insurance coverage stored in the TYPE attributes without showing any repeated values . This is achieved using the SQL keyword DISTINCT .
- Query3: To get the city and staff names we used a JOIN operation to combine data from the Staff,Hospital,Department and Workin tables based on the IDs.
- Query4: To find appointments scheduled for a specific interval, we used a WHERE clause to filter the results based on the appointment’s Status and the function CURDATE() + INTERVAL 7 DAY.
- Query5: We used the function GROUP BY to group the results by department name and COUNT to count the number of appointments in each department.
- Query6: We used a JOIN operation to combine data from the Hospital and Stock tables. Then we used GROUP BY to calculate the average UnitPrice for each hospital.

- Query7: We used the COUNT function to calculate the total emergencies for every hospital. After that we used HAVING clause that filters the counted groups to show only those hospitals where the total count is greater than 20 .
- Query8: We first selected the medicaments that have the therapeutic class antibiotic ,then looked for those that cost less than 200 using the statement IN since the UnitPrice is in the Stock table.
- Query9: We used a normal WHERE clause to filter the UnitPrice for the medicaments that are the most expensive(normal comparison between the values within the table stock S and stock S2).
- Query10: This query was similar to query 5 but instead of counting total appointments we calculated them based on their status (completed-scheduled-cancelled). And for that we used the CASE statement within the COUNT function.
- Query11: We used a JOIN operation to combine data from the Patient, Appointment and ClinicalActivity tables. Then we selected only the rows where the date of the next appointment is more than 30 days from the current date using the function CURDATE() + INTERVAL 30 DAY.
- Query12: We understand from the query 12 that we have two objectives:
 - First: Compute the total number of appointments .
 - Second: Compute the percentage share of appointments for the staff in his own hospital .

To achieve this we used a subquery within the SELECT statements that calculate the total number of appointments for the entire hospital by linking it to the staff by the outer query using HID . Finally the percentage share is calculated using the formula $\text{Percentage_share} = (\text{staff_appointments_total} / \text{hospital_appointments_total}) * 100$.

- Query13: For this query we tried to look for all the possible tuples that verify the conditions mentioned in the query . So we selected all the hospitals that have at least one drug that is out of stock.
- Query14: We first selected the IDs of that have all the antibiotics medicaments in stock. Then we used NOT EXISTS clause to find the hospitals that have all the antibiotics medicaments in stock.
- Query15: First we selected the average UnitPrice of the whole city hospitals using the function GROUP BY the TherapeuticClass . Then we joined this selection with the Medication and Stock tables to count the average by each hospital. Finally we used the clause CASE WHEN to flag the cases where the average UnitPrice in each hospital is higher or lower than the city average.
- Query16: We selected the minimum date of scheduled appointments of each patient using the function MIN along with GROUP BY PatientID. Then we joined this selection with the Patient, Appointment and ClinicalActivity tables to get their earliest appointment dates.

- Query17: We understand from the query 17 that we need to select patients based on two criteria applied to their emergency visits . First , we count the total number of emergency visits for each patient and use the having clause to filter only those whose counts are greater than 2 . After that we calculate the date of the latest visits and verify that this date falls within the last fourteen days . finally we group these two conditions using the logical operator AND.
- Query18: We first inner joined the Hospital,Department,ClinicalAcitivity and Ap-
pointment tables to get all the necessary data .Then we grouped the results by
hospital city and used COUNTto calculate how many completed appointmentseach
hospital has .Finally we used RANK() window function to rank hospitals in each
city based on the number of completed appointments in descending order in the
last 90 days.
- Query19: For this query, we first used a JOIN operation to combine the Stock and
Hospital tables in order to retrieve the medication ID (MID) along with the city
it belongs to. Then, in the subquery, we grouped the Stock data by medication
and city using GROUP BY, and we applied the functions MAX() and MIN() to
calculate the price variation of each medication across hospitals. Finally, we filtered
the results to include only those medications where the price variation exceeds 30
percent and used the IN statment to return only those medications from the main
query whose price variability meets this condition.
- Query20: This query only checks the data quality by ensuring that the UnitPrice
and Qty are both positive values.So we used a simple WHERE clause to filter any
negative values within the Stock table.

4 -Implementation

4.1 -The refined MNHS Database Schema

```

1  CREATE DATABASE MNHS;
2  USE MNHS;
3
4  CREATE TABLE Patient (
5      IID INT PRIMARY KEY,
6      CIN VARCHAR(10) UNIQUE NOT NULL,
7      FullName VARCHAR(100) NOT NULL,
8      Birth DATE,
9      Sex ENUM('M', 'F') NOT NULL,
10     BloodGroup ENUM('A+', 'A-', 'B+', 'B-', 'O+', 'O-',
11                     'AB+', 'AB-'),
12     Phone VARCHAR(15)
13 );
14
15 CREATE TABLE Hospital (
16     HID INT PRIMARY KEY,
17     Name VARCHAR(100) NOT NULL,
18     City VARCHAR(50) NOT NULL,

```

```
18     Region VARCHAR(50)
19 );
20
21 CREATE TABLE Department (
22     DEP_ID INT,
23     HID INT,
24     Name VARCHAR(100) NOT NULL,
25     Specialty VARCHAR(100),
26     PRIMARY KEY (DEP_ID),
27     FOREIGN KEY (HID) REFERENCES Hospital(HID)
28 );
29
30 CREATE TABLE Staff (
31     STAFF_ID INT PRIMARY KEY,
32     FullName VARCHAR(100) NOT NULL,
33     Status ENUM('Active', 'Retired') DEFAULT 'Active'
34 );
35
36 CREATE TABLE Workin (
37     STAFF_ID INT,
38     DEP_ID INT,
39     primary key(STAFF_ID, DEP_ID),
40     FOREIGN KEY (STAFF_ID) REFERENCES Staff(STAFF_ID),
41     FOREIGN KEY (DEP_ID) REFERENCES Department(DEP_ID)
42 );
43
44 CREATE TABLE ClinicalActivity (
45     CAID INT PRIMARY KEY,
46     IID INT NOT NULL,
47     STAFF_ID INT NOT NULL,
48     DEP_ID INT NOT NULL,
49     Date DATE NOT NULL,
50     Time TIME,
51     FOREIGN KEY (IID) REFERENCES Patient(IID),
52     FOREIGN KEY (STAFF_ID) REFERENCES Staff(STAFF_ID),
53     FOREIGN KEY (DEP_ID) REFERENCES Department(DEP_ID)
54 );
55
56 CREATE TABLE Appointment (
57     CAID INT PRIMARY KEY,
58     Reason VARCHAR(100),
59     Status ENUM('Scheduled', 'Completed', 'Cancelled') DEFAULT '
60     Scheduled',
61     FOREIGN KEY (CAID) REFERENCES ClinicalActivity(CAID)
62 );
63
64 CREATE TABLE Emergency (
65     CAID INT PRIMARY KEY,
66     TriageLevel INT CHECK (TriageLevel BETWEEN 1 AND 5),
67     Outcome ENUM('Discharged', 'Admitted', 'Transferred', '
68     Deceased'),
```

```

67     FOREIGN KEY (CAID) REFERENCES ClinicalActivity(CAID)
68 );
69
70 CREATE TABLE Insurance (
71     INS_ID INT PRIMARY KEY,
72     Type ENUM('CNOPS', 'CNSS', 'RAMED', 'Private', 'None') NOT
        NULL
73 );
74
75
76 CREATE TABLE Expense (
77     EXP_ID INT PRIMARY KEY,
78     INS_ID INT,
79     CAID INT UNIQUE NOT NULL,
80     Total DECIMAL(10, 2) NOT NULL CHECK (Total >= 0),
81     FOREIGN KEY (INS_ID) REFERENCES Insurance(INS_ID),
82     FOREIGN KEY (CAID) REFERENCES ClinicalActivity(CAID)
83 );
84
85 CREATE TABLE Medication (
86     MID INT PRIMARY KEY,
87     Name VARCHAR(100) NOT NULL,
88     Form VARCHAR(50),
89     Strength VARCHAR(50),
90     ActiveIngredient VARCHAR(100),
91     TherapeuticClass VARCHAR(100),
92     Manufacturer VARCHAR(100)
93 );
94
95 CREATE TABLE Stock (
96     HID INT,
97     MID INT,
98     StockTimestamp DATETIME DEFAULT CURRENT_TIMESTAMP,
99     UnitPrice DECIMAL(10, 2) CHECK (UnitPrice >= 0),
100    Qty INT DEFAULT 0 CHECK (Qty >= 0),
101    ReorderLevel INT DEFAULT 10 CHECK (ReorderLevel >= 0),
102    PRIMARY KEY (HID, MID, StockTimestamp),
103    FOREIGN KEY (HID) REFERENCES Hospital(HID),
104    FOREIGN KEY (MID) REFERENCES Medication(MID)
105 );
106
107 CREATE TABLE Prescription (
108     PID INT PRIMARY KEY,
109     CAID INT UNIQUE NOT NULL,
110     DateIssued DATE NOT NULL,
111     FOREIGN KEY (CAID) REFERENCES ClinicalActivity(CAID)
112 );
113
114 -- Many to many relationship between prescription and medication
    entities
115 CREATE TABLE Includes (

```

```

116     PID INT,
117     MID INT,
118     Dosage VARCHAR(100),
119     Duration VARCHAR(50),
120     PRIMARY KEY (PID, MID),
121     FOREIGN KEY (PID) REFERENCES Prescription(PID),
122     FOREIGN KEY (MID) REFERENCES Medication(MID)
123 );
124
125 CREATE TABLE ContactLocation (
126     CLID INT PRIMARY KEY,
127     City VARCHAR(50),
128     Province VARCHAR(50),
129     Street VARCHAR(100),
130     Number VARCHAR(10),
131     PostalCode VARCHAR(10),
132     PhoneLocation VARCHAR(15)
133 );
134
135 -- Many to many relationship between Patient and Contact Location
    entities
136 CREATE TABLE have (
137     IID INT,
138     CLID INT,
139     PRIMARY KEY (IID, CLID),
140     FOREIGN KEY (IID) REFERENCES Patient(IID),
141     FOREIGN KEY (CLID) REFERENCES ContactLocation(CLID)
142 );

```

4.2 -Altering tables

```

1 -- Altering patient table to add email attribute
2 ALTER TABLE Patient
3 ADD Email VARCHAR(100)
4 CHECK (Email LIKE '_%@_%._%');
5
6
7 -- Altering Staff table to add address column
8 ALTER TABLE Staff
9 ADD Address VARCHAR(50) ;

```

4.3 -Inserting data

```

1 -- Inserting sample rows
2 -- Patient
3 INSERT INTO Patient (IID, CIN, FullName, Birth, Sex, BloodGroup,
4     Phone, Email)
5 VALUES

```

```

5 (1, 'CA123456', 'Youssef El Idrissi', '1985-04-12', 'M', 'A+', '
6 0612345678', 'y.elidrissi@casablanca.ma'),
7 (2, 'RA789012', 'Fatima Zahra Bennani', '1990-09-25', 'F', 'O-',
8 '0654321987', 'f.zahra@rabat.ma'),
9 (3, 'ME345678', 'Mohamed Amrani', '1978-01-30', 'M', 'B+', '
10 0623456789', 'm.amrani@meknes.ma'),
11 (4, 'FE901234', 'Khadija El Fassi', '2000-06-15', 'F', 'AB+', '
12 0678912345', 'k.elfassi@fes.ma'),
13 (5, 'MA567890', 'Rachid Toumi', '1995-03-10', 'M', 'O+', '
14 0645789123', 'r.toumi@marrakech.ma');
15
16 -- Hospital
17 INSERT INTO Hospital (HID, Name, City, Region)
18 VALUES
19 (1, 'CHU Hassan II', 'Casablanca', 'Casablanca-Settat'),
20 (2, 'H pital Ibn Sina', 'Rabat', 'Rabat-Sal -K nitra'),
21 (3, 'H pital Mohammed V', 'Marrakech', 'Marrakesh-Safi'),
22 (4, 'H pital Regional Tanger', 'Tangier', 'Tanger-Tetouan-Al
23 Hoceima'),
24 (5, 'Centre Hospitalier Universitaire F s', 'F s', 'F s -
25 Mekn s');
26
27 -- Department
28 INSERT INTO Department (DEP_ID, HID, Name, Specialty) VALUES
29 (1, 1, 'Cardiology', 'Heart diseases'),
30 (2, 2, 'Neurology', 'Brain and nerves'),
31 (3, 3, 'Oncology', 'Cancer treatment'),
32 (4, 4, 'Pediatrics', 'Child healthcare'),
33 (5, 5, 'Orthopedics', 'Bone and muscles');
34
35 -- Staff
36 INSERT INTO Staff (STAFF_ID, FullName, Status, Address) VALUES
37 (1, 'Dr. Salma El Alami', 'Active', '12 Boulevard Mohammed V,
38 Casablanca'),
39 (2, 'Dr. Yassine Najjar', 'Active', '45 Rue Al Qods, Rabat'),
40 (3, 'Dr. Amina Boulahlib', 'Active', '3 Jamaa el Fna Sq,
41 Marrakech'),
42 (4, 'Dr. Rami Sedrati', 'Retired', '8 Medina St, Tangier'),
43 (5, 'Dr. Laila Chraibi', 'Active', '20 Andalus Ave, F s');
44
45 -- Workin
46 INSERT INTO Workin (STAFF_ID, DEP_ID)
47 VALUES
48 (1, 1),
49 (2, 2),
50 (3, 3),
51 (4, 4),
52 (5, 5);
53
54 -- ClinicalActivity

```



```

47 INSERT INTO ClinicalActivity (CAID, IID, STAFF_ID, DEP_ID, Date,
    Time)
48 VALUES
49 (1, 1, 1, 1, '2025-11-10', '09:30:00'),
50 (2, 2, 2, 2, '2025-11-11', '14:00:00'),
51 (3, 3, 3, 3, '2025-11-12', '10:15:00'),
52 (4, 4, 4, 4, '2025-11-13', '16:45:00'),
53 (5, 5, 5, 5, '2025-11-14', '08:00:00');
54
55 -- Appointment
56 INSERT INTO Appointment (CAID, Reason, Status)
57 VALUES
58 (1, 'Routine checkup', 'Completed'),
59 (2, 'Headache evaluation', 'Scheduled'),
60 (3, 'Cancer screening', 'Completed'),
61 (4, 'Vaccination', 'Cancelled'),
62 (5, 'Bone fracture follow-up', 'Completed');
63
64 -- Emergency
65 INSERT INTO Emergency (CAID, TriageLevel, Outcome)
66 VALUES
67 (1, 3, 'Discharged'),
68 (2, 2, 'Admitted'),
69 (3, 4, 'Transferred'),
70 (4, 1, 'Deceased'),
71 (5, 5, 'Discharged');
72
73 -- Insurance
74 INSERT INTO Insurance (INS_ID, Type)
75 VALUES
76 (1, 'CNOPS'),
77 (2, 'CNSS'),
78 (3, 'RAMED'),
79 (4, 'Private'),
80 (5, 'None');
81
82 -- Expense
83 INSERT INTO Expense (EXP_ID, INS_ID, CAID, Total)
84 VALUES
85 (1, 1, 1, 1200.50),
86 (2, 2, 2, 800.00),
87 (3, 3, 3, 1500.75),
88 (4, 4, 4, 500.25),
89 (5, 5, 5, 300.00);
90
91 -- Medication
92 INSERT INTO Medication (MID, Name, Form, Strength,
    ActiveIngredient, TherapeuticClass, Manufacturer)
93 VALUES
94 (1, 'Paracetamol', 'Tablet', '500mg', 'Paracetamol', 'Analgesic',
    'Pharma Maroc'),

```

```

95 (2, 'Amoxicillin', 'Capsule', '250mg', 'Amoxicillin', 'Antibiotic
    ', 'Sandoz Maroc'),
96 (3, 'Ibuprofen', 'Tablet', '400mg', 'Ibuprofen', 'Anti-
    inflammatory', 'Rameda'),
97 (4, 'Omeprazole', 'Capsule', '20mg', 'Omeprazole', 'Proton pump
    inhibitor', 'Medpharm Maroc'),
98 (5, 'Cetirizine', 'Tablet', '10mg', 'Cetirizine', 'Antihistamine'
    , 'Pharma Plus');
99
100
101
102 -- Stock
103 INSERT INTO Stock (HID, MID, StockTimestamp, UnitPrice, Qty,
    ReorderLevel)
104 VALUES
105 (1, 1, '2025-11-01 08:00:00', 0.5, 100, 20),
106 (2, 2, '2025-11-02 09:00:00', 1.2, 50, 10),
107 (3, 3, '2025-11-03 10:00:00', 0.8, 75, 15),
108 (4, 4, '2025-11-04 11:00:00', 1.5, 30, 5),
109 (5, 5, '2025-11-05 12:00:00', 0.7, 90, 25);
110
111 -- Prescription
112 INSERT INTO Prescription (PID, CAID, DateIssued)
113 VALUES
114 (1, 1, '2025-11-10'),
115 (2, 2, '2025-11-11'),
116 (3, 3, '2025-11-12'),
117 (4, 4, '2025-11-13'),
118 (5, 5, '2025-11-14');
119
120 -- Includes
121 INSERT INTO Includes (PID, MID, Dosage, Duration)
122 VALUES
123 (1, 1, '500mg twice a day', '5 days'),
124 (2, 2, '250mg three times a day', '7 days'),
125 (3, 3, '400mg once a day', '3 days'),
126 (4, 4, '20mg once a day', '14 days'),
127 (5, 5, '10mg once a day', '10 days');
128
129 -- ContactLocation
130 INSERT INTO ContactLocation (CLID, City, Province, Street, Number
    , PostalCode, PhoneLocation) VALUES
131 (1, 'Casablanca', 'Casablanca-Settat', 'Mohammed V Blvd', '12', '
    20000', '0612345678'),
132 (2, 'Rabat', 'Rabat-Sal -K nitra', 'Al Qods St', '45', '10000',
    '0654321987'),
133 (3, 'Marrakech', 'Marrakesh-Safi', 'Jamaa el Fna Sq', '3', '40000
    ', '0623456789'),
134 (4, 'Tangier', 'Tanger-Tetouan-Al Hoceima', 'Medina St', '8', '
    90000', '0678912345'),

```

```

135 (5, 'F s', 'F s-Mekn s', 'Andalus Ave', '20', '30000', '
    0645789123');
136
137 -- have
138 INSERT INTO have (IID, CLID)
139 VALUES
140 (1, 1),
141 (2, 2),
142 (3, 3),
143 (4, 4),
144 (5, 5);

```

4.4 -Updating data

```

1 -- Updating a patients phone number
2 UPDATE Patient
3 SET Phone = '0666114696'
4 WHERE IID = 5;

```

4.5 -Modifying data

```

1 -- Modifying a Hospitals region
2 UPDATE Hospital
3 SET City = 'Settat',
4     Region = 'Casablanca-Settat'
5 WHERE HID = 2;

```

4.6 -Deleting data

```

1 --Deleting a cancelled appointment
2 DELETE FROM Appointment
3 WHERE Status = 'Cancelled';

```

4.7 -Use of normalization

Normalization was applied to the MNHS database schema to ensure that it adheres to the Boyce-Codd Normal Form (BCNF). This process involved analyzing the functional dependencies within the database and decomposing relations that violated BCNF into smaller, well-structured relations. The goal was to eliminate redundancy, prevent update anomalies, and maintain data integrity. Each relation was carefully examined to ensure that every determinant is a candidate key, thereby achieving BCNF compliance.

4.8 -Queries script and execution

- QUERY 1:

```
1 SELECT FullName ,
2 SUBSTRING_INDEX(FullName , ',' , -1) as last_name
3 FROM Patient
4 ORDER BY SUBSTRING_INDEX(FullName , ',' , -1);
```

	FullName	last_name
►	Fatima Zahra Bennani	Fatima Zahra Bennani
	Khadija El Fassi	Khadija El Fassi
	Mohamed Amrani	Mohamed Amrani
	Rachid Toumi	Rachid Toumi
	Youssef El Idrissi	Youssef El Idrissi

Figure 1: Query 1 Execution Result

• QUERY 2:

```
1 SELECT DISTINCT Type
2 FROM Insurance ;
```

	Type
►	CNOPS
	CNSS
	RAMED
	Private
	None

Figure 2: Query 2 Execution Result

• QUERY 3:

```
1 SELECT S.Fullname
2 FROM Staff S, Hospital H , Departement D , Workin W
3 WHERE S.STAFF_ID = W.STAFF_ID
4 AND W.DEP_ID = D.DEP_ID
5 AND D.HID = H.HID
6 AND H.City = 'Rabat';
```

FullName

Figure 3: Query 3 Execution Result

• QUERY 4:

```

1 SELECT DISTINCT A.CAID
2 FROM Appointment A
3 WHERE A.Status = 'Scheduled'
4     AND EXISTS (
5         SELECT Ca.CAID
6         FROM ClinicalActivity Ca
7         WHERE Ca.CAID = A.CAID
8             AND Ca.Date <= CURDATE() + INTERVAL 7 DAY
9     );

```

	CAID
▶	2
●	NULL

Figure 4: Query 4 Execution Result

• QUERY 5:

```

1 SELECT d.Name,d.DEP_ID
2 COUNT(a.CAID) AS nbr_appt
3 FROM Department d
4 JOIN ClinicalActivity ca ON ca.DEP_ID=d.DEP_ID
5 JOIN Appointment a ON a.CAID=ca.CAID
6 GROUP BY d.Name,d.DEP_ID;

```

	Name	DEP_ID	nbr_appt
▶	Cardiology	1	1
	Neurology	2	1
	Oncology	3	1
	Orthopedics	5	1

Figure 5: Query 5 Execution Result

• QUERY 6:

```

1 SELECT HID,AVG(UnitPrice) as average_Unit_price
2 FROM Stock S
3     NATURAL JOIN Hospital H
4 GROUP BY HID;

```

	HID	average_Unit_price
	1	0.500000
	2	1.200000
	3	0.800000
	4	1.500000
	5	0.700000

Figure 6: Query 6 Execution Result

• QUERY 7:

```

1 SELECT
2     H.Name ,
3     COUNT(E.CAID) AS EmergencyCount
4 FROM
5     Hospital H
6 JOIN
7     Department D ON D.HID = H.HID
8 JOIN
9     ClinicalActivity C ON C.DEP_ID = D.DEP_ID
10 JOIN
11     Emergency E ON E.CAID = C.CAID
12 GROUP BY
13     H.HID, H.Name
14 HAVING
15     COUNT(E.CAID) > 20;

```

	Name	EmergencyCount
--	------	----------------

Figure 7: Query 7 Execution Result

• QUERY 8:

```

1 SELECT M.Name
2 FROM Medication M
3 WHERE M.TherapeuticClass = 'Antibiotic'
4     AND M.Name IN (SELECT M2.Name
5                     FROM Stock S , Medication M2
6                     WHERE S.MID = M2.MID
7                     AND S.UnitPrice <= 200);

```

	Name
▶	Amoxicillin

Figure 8: Query 8 Execution Result

• QUERY 9:

```

1 SELECT S.HID , S.MID
2 FROM Stock S
3 WHERE (
4     SELECT COUNT(*)
5     FROM Stock S2
6     WHERE S2.HID = S.HID
7         AND S2.UnitPrice > S.UnitPrice
8 ) < 3;

```

	HID	MID
▶	1	1
	2	2
	3	3
	4	4
	5	5

Figure 9: Query 9 Execution Result

• QUERY 10:

```

1 SELECT d.Name ,d.DEP_ID ,
2 COUNT(CASE WHEN a.Status='Scheduled' THEN 1 END) AS
   nbr_scheduled ,
3 COUNT(CASE WHEN a.Status='Completed' THEN 1 END) AS
   nbr_completed ,
4 COUNT(CASE WHEN a.Status='Cancelled' THEN 1 END) AS
   nbr_cancelled
5 FROM Department d
6 JOIN ClinicalActivity ca ON ca.DEP_ID=d.DEP_ID
7 JOIN Appointment a ON a.CAID=ca.CAID
8 GROUP BY d.Name ,d.DEP_ID;

```

	Name	DEP_ID	nbr_scheduled	nbr_completed	nbr_cancelled
►	Cardiology	1	0	1	0
	Neurology	2	1	0	0
	Oncology	3	0	1	0
	Orthopedics	5	0	1	0

Figure 10: Query 10 Execution Result

• QUERY 11:

```

1 SELECT *
2 FROM Patient P
3   NATURAL JOIN Appointment A
4   NATURAL JOIN ClinicalActivity CA
5 where CA.DATE > CURDATE() + INTERVAL 30 DAY;
```

IID	CAID	CIN	FullName	Birth	Sex	BloodGroup	Phone	Email	Reason	Status	STAFF_ID	DEP_ID	Date	Time
-----	------	-----	----------	-------	-----	------------	-------	-------	--------	--------	----------	--------	------	------

Figure 11: Query 11 Execution Result

• QUERY 12:

```

1 SELECT
2     S.FullName ,
3     H.Name ,
4     COUNT(A.CAID) AS total_staff_appointment ,
5     (
6         SELECT COUNT(A2.CAID)
7         FROM Appointment A2
8         JOIN ClinicalActivity CA2 ON A2.CAID = CA2.CAID
9         JOIN Department D2 ON CA2.DEP_ID = D2.DEP_ID
10        WHERE D2.HID = H.HID
11    ) AS hospital_appointments_total ,
12    (
13        COUNT(A.CAID) * 100.0 / (
14            SELECT COUNT(A2.CAID)
15            FROM Appointment A2
16            JOIN ClinicalActivity CA2 ON A2.CAID = CA2.CAID
17            JOIN Department D2 ON CA2.DEP_ID = D2.DEP_ID
18            WHERE D2.HID = H.HID
19        )
20    ) AS percentage_share
21 FROM Staff S
22 JOIN ClinicalActivity CA ON S.STAFF_ID = CA.STAFF_ID
23 JOIN Appointment A ON CA.CAID = A.CAID
24 JOIN Department D ON CA.DEP_ID = D.DEP_ID
```



```

25 JOIN Hospital H ON D.HID = H.HID
26 GROUP BY S.STAFF_ID, S.FullName, H.HID, H.Name
27 ORDER BY H.Name, percentage_share DESC;

```

	FullName	Name	total_staff_appointment	hospital_appointments_total	percentage_share
▶	Dr. Laila Chraïbi	Centre Hospitalier Universitaire Fès	1	1	100.00000
	Dr. Salma El Alami	CHU Hassan II	1	1	100.00000
	Dr. Yassine Najjar	Hôpital Ibn Sina	1	1	100.00000
	Dr. Amina Boulahlib	Hôpital Mohammed V	1	1	100.00000

Figure 12: Query 12 Execution Result

• QUERY 13:

```

1 SELECT M.Name , H.Name
2 FROM Medication M , Hospital H
3 WHERE EXISTS (SELECT S.MID
4                FROM Stock S
5                WHERE S.mid = M.mid
6                      AND S.HID = H.HID
7                      AND S.Qty < S.ReorderLevel );

```

	Name	Name

Figure 13: Query 13 Execution Result

• QUERY 14:

```

1 SELECT S.HID
2 FROM Stock S
3 WHERE NOT EXISTS (
4     SELECT M.MID
5     FROM Medication M
6     WHERE M.TherapeuticClass = 'Antibiotic'
7     AND NOT EXISTS (
8         SELECT 1
9         FROM Stock S2
10        WHERE S2.HID = S.HID
11              AND S2.MID = M.MID
12     )
13 );

```

	HID
▶	2

Figure 14: Query 14 Execution Result

• QUERY 15:

```

1 SELECT s.HID,m.TherapeuticClass ,
2     AVG(s.UnitPrice) AS avg_price,
3     CASE
4         WHEN AVG(s.UnitPrice)>c.avg_city THEN 'Above'
5         WHEN AVG(s.UnitPrice)<c.avg_city THEN 'Below'
6         WHEN AVG(s.UnitPrice)=c.avg_city THEN 'Equal'
7     END AS avg_flag
8 FROM Medication m
9 JOIN Stock s ON m.MID=s.MID
10 JOIN (SELECT m.TherapeuticClass ,
11           AVG(UnitPrice) AS avg_city
12         FROM Medication m
13         JOIN Stock s ON m.MID=s.MID
14         GROUP BY m.TherapeuticClass) c ON m.
           TherapeuticClass=c.TherapeuticClass
15 GROUP BY s.HID,m.TherapeuticClass ,c.avg_city;
```

	HID	TherapeuticClass	avg_price	avg_flag
▶	1	Analgesic	0.500000	Equal
	2	Antibiotic	1.200000	Equal
	3	Anti-inflammatory	0.800000	Equal
	4	Proton pump inhibitor	1.500000	Equal
	5	Antihistamine	0.700000	Equal

Figure 15: Query 15 Execution Result

• QUERY 16:

```

1 SELECT MIN(Date) AS next_a, IID
2 FROM Patient
3 NATURAL JOIN Appointment
4 NATURAL JOIN ClinicalActivity
5 WHERE Status = 'Scheduled'
6 GROUP BY IID
7 ORDER BY next_a;
```

	next_a	IID
▶	2025-11-11	2

Figure 16: Query 16 Execution Result

• QUERY 17:

```

1 SELECT
2     P.FullName ,
3     COUNT(E.CAID) AS total_visits ,
4     MAX(C.Date) AS recent_date
5 FROM
6     Patient P
7 JOIN
8     ClinicalActivity C ON P.IID = C.IID
9 JOIN
10    Emergency E ON C.CAID = E.CAID
11 GROUP BY
12     P.IID, P.FullName
13 HAVING
14     COUNT(E.CAID) >= 2
15     AND MAX(C.Date) >= '2025-11-01'
16 ORDER BY
17     P.FullName;

```

	FullName	total_visits	recent_date
--	----------	--------------	-------------

Figure 17: Query 17 Execution Result

• QUERY 18:

```

1 SELECT H.City ,
2         H.Name ,
3         COUNT(A.CAID) AS count ,
4         RANK() OVER (PARTITION BY H.City
5                       ORDER BY COUNT(A.CAID) DESC) AS rank
6 From Hospital H
7 INNER JOIN Department D ON H.HID = D.HID
8 INNER JOIN ClinicalActivity CA ON D.DEP_ID = CA.DEP_ID
9 INNER JOIN Appointment A ON A.CAID = CA.CAID
10 WHERE A.Status = 'Completed'
11        AND CA.DATE >= DATE_SUB(CURRENT_DATE, INTERVAL 90 DAY)

```

```
12 GROUP BY H.City , H.HID , H.Name
13 ORDER BY H.City , rank ;
```

	City	Name	total_completed	rank_value
▶	Casablanca	CHU Hassan II	1	1
	Fès	Centre Hospitalier Universitaire Fès	1	1
	Marrakech	Hôpital Mohammed V	1	1

Figure 18: Query 18 Execution Result

• QUERY 19:

```
1 SELECT S.MID, H.City
2 FROM Stock S
3 JOIN Hospital H ON S.HID = H.HID
4 WHERE S.MID IN (
5     SELECT S2.MID
6     FROM Stock S2
7     JOIN Hospital H2 ON S2.HID = H2.HID
8     GROUP BY S2.MID, H2.City
9     HAVING (MAX(S2.UnitPrice) - MIN(S2.UnitPrice)) / MIN(S2.
10 UnitPrice) > 0.3
);
```

Result Grid			
	MID	City	

Figure 19: Query 19 Execution Result

• QUERY 20:

```
1 SELECT *
2 FROM Stock s
3 WHERE Qty<0 OR UnitPrice<=0;
```

	HID	MID	StockTimestamp	UnitPrice	Qty	ReorderLevel
*	NULL	NULL	NULL	NULL	NULL	NULL

Figure 20: Query 20 Execution Result

5 -Discussion

In this Lab , one of the main challenges we faced was ensuring that our database schema was properly normalized to eliminate redundancy and maintain data integrity. We had to carefully analyze the functional dependencies. Another challenge was writing complex SQL queries that involved multiple joins, subqueries, and aggregate functions to retrieve the required information accurately. We overcame these challenges by thoroughly reviewing database design principles and practicing SQL query writing and researching new functions and clauses that could be useful in the queries script. Additionally, we collaborated as a team to discuss and resolve any issues that arose during the implementation process.

6 -Conclusion

This lab provided us with experience in database design, normalization, and SQL query writing. We learned how to create a well structured database schema that adheres to normalization principles, ensuring data integrity and reducing redundancy. The process of writing and executing complex SQL queries helped us enhance our skills in data retrieval and manipulation. Overall, this lab has strengthened our understanding of database management systems and prepared us for more advanced projects.